

1. Overview of Data Governance and Unity Catalog

Data Governance

Data Governance is the process of managing data availability, usability, security, quality, and compliance within an organization.

Objectives

- Ensure data security
- Maintain data quality
- Control data access
- Meet regulatory compliance
- Enable data discovery and sharing

Key Components

- **Data Security** – Protect data from unauthorized access
- **Access Control** – Define who can access data
- **Data Lineage** – Track data flow from source to destination
- **Data Auditing** – Monitor data usage and changes
- **Metadata Management** – Store information about data

Unity Catalog

Unity Catalog is Databricks' unified governance solution for managing data and AI assets across workspaces.

Features of Unity Catalog

- Centralized access control
- Fine-grained permissions
- Data lineage tracking
- Data discovery and search
- Audit logging
- Governance across multiple workspaces

Benefits

- Single governance model
- Enhanced security
- Simplified administration
- Better compliance management

2. Create Unity Catalog Metastore and Enable a Databricks Workspace with Unity Catalog

What is a Metastore?

A **Metastore** is the top-level container in Unity Catalog that stores metadata about catalogs, schemas, tables, views, and permissions.

Steps to Create a Metastore

Step 1: Open Account Console

- Log in to your Databricks Account Console.
- Navigate to **Data** → **Metastores**.

Step 2: Create Metastore

- Click **Create Metastore**.
- Enter:
 - Metastore Name
 - Region
 - Storage Location
- Click **Create**.

Step 3: Assign Workspace

- Select the created metastore.
- Click **Assign to Workspace**.
- Choose the Databricks workspace.
- Save changes.

Verify Unity Catalog

Run:

SHOW CATALOGS;

If Unity Catalog is enabled, catalogs such as:

main

samples

will be displayed.

Check Current Catalog

```
SELECT current_catalog();
```

Benefits of Enabling Unity Catalog

- Centralized governance
- Shared metadata
- Consistent permissions
- Cross-workspace access

3. Overview of 3-Level Namespace and Creating Unity Catalog Objects

3-Level Namespace

Unity Catalog organizes data using:

catalog.schema.table

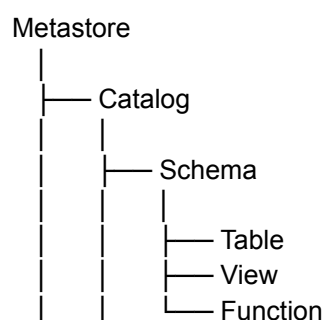
Example:

Sales.customer.orders

where

Level	Description
Catalog	Top-level container
Schema	Database inside catalog
Table	Data object

Hierarchy



Create Catalog

CREATE CATALOG sales;

View catalogs:

SHOW CATALOGS;

Use Catalog

USE CATALOG sales;

Create Schema

CREATE SCHEMA customer;

View schemas:

SHOW SCHEMAS;

Use Schema

USE SCHEMA customer;

Create Table

**CREATE TABLE orders (
 order_id INT,
 customer_name STRING,
 amount DOUBLE
);**

Insert Data

**INSERT INTO orders
VALUES
(1,'Rahul',5000),
(2,'Priya',7500);**

Query Table

SELECT * FROM orders;

Create View

**CREATE VIEW high_value_orders AS
SELECT *
FROM orders
WHERE amount > 6000;**

Fully Qualified Name

sales.customer.orders

Advantages of 3-Level Namespace

- Better organization
- Strong governance
- Easier access control
- Multi-team data sharing
- Reduced naming conflicts

Summary

Object	Example
Metastore	Company-wide metadata container
Catalog	sales
Schema	customer
Table	orders
Full Path	sales.customer.orders

Namespace Format:

catalog.schema.table

This is the fundamental structure used by Unity Catalog for organizing and governing data assets in Databricks.

so metastore is the main container
which enables centralized governance and fine grained access control
while the unit catalog is the heirarchical approach through which we achive it in
databricks